Selenium with JavaScript: compact cheatsheet

STEP 1 Meet Selenium

1.1 What Selenium is and where it came from

What Selenium is An umbrella project of tools and libraries that automate real browsers the way a user would. At its core, a browser user agent library.

Three parts WebDriver (library plus browser driver, for code), IDE (record and play back extension), Grid (many sessions on many machines).

Not a test framework Selenium drives the browser. Assertions, test structure and reports come from JUnit, pytest, Mocha and similar tools.

Key dates Selenium 2.0 on 8 July 2011, 3.0 on 13 October 2016, W3C Recommendation on 5 June 2018, Selenium 4.0 on 13 October 2021.

Program shape Start a session, send the browser somewhere, ask or act, and always quit at the end.

```
(async function firstLook() {
```

Every Selenium call in JavaScript returns a promise, so we work inside an async function.

```
const driver = await new Builder().forBrowser(Browser.CHROME).build();
```

Build a driver for Chrome. Selenium Manager finds chromedriver and a session starts.

```
await driver.get('https://www.selenium.dev/selenium/web/web-form.html');
```

Open the page and wait for it to load.

TRAP Selenium only controls the browser. It does not decide whether a test passed, it does not write reports and it does not give you Given, When, Then syntax. Those come from a test framework such as JUnit, pytest or Mocha. Chapter 5.1 joins the two.

1.2 Architecture and the W3C WebDriver protocol

Three programs Your code and the binding (local end), the driver (remote end), the browser. The driver is written by the browser vendor.

HTTP and JSON Every command is one HTTP request with a fixed method and path. Answers are {"value": ...} on success or {"value": {"error", "message", "stacktrace"}} on failure.

Session id POST /session returns a session id. Every later path contains it. DELETE /session/{id} ends it.

Element reference Elements travel as {"element-6066-11e4-a52e-4f735466cecf": "id"}. Bindings turn id, name and class name into CSS selectors.

BiDi Classic WebDriver is request and response. WebDriver BiDi adds a WebSocket so the browser can send events. CDP access is temporary until BiDi covers it.

```
async function call(method, path, body) {
```

Send one W3C command and return the part of the answer under "value".

```
const res = await fetch('http://localhost:9515' + path, {
```

fetch is built into Node.js 18 and later.

```
body: body === undefined ? undefined : JSON.stringify(body),
```

POST bodies are JSON objects. GET and DELETE have none.

```
const driver = spawn('chromedriver', ['--port=9515'], { stdio: 'ignore' });
```

Start the driver yourself and hide its own messages.

```
await new Promise(r => setTimeout(r, 1000));
```

Give it a moment to open its port.

```
const KEY = 'element-6066-11e4-a52e-4f735466cecf';
```

The fixed key for element references.

```
await call('POST', s + '/timeouts', { implicit: 2000 });
```

Set Timeouts: Find Element may keep looking for 2 seconds while the next page loads.

```
await call('POST', s + '/url', { url: 'https://www.selenium.dev/selenium/web/web-form.html' });
```

Navigate To.

```
await call('GET', s + '/title');
```

Get Title.

```
const box = (await call('POST', s + '/element', { using: 'css selector', value: "[name='my-text']" }))[KEY];
```

Find Element.

```
await call('POST', s + '/element/' + box + '/value', { text: 'Selenium' });
```

Element Send Keys.

```
await call('POST', s + '/element/' + button + '/click', {});
```

Element Click.

```
await call('GET', s + '/element/' + message + '/text');
```

Get Element Text.

```
await call('DELETE', s);
```

Delete Session.

```
driver.kill();
```

Stop chromedriver.

Success HTTP 200 and {"value": result}

Error HTTP 4xx or 5xx and {"value": {"error", "message", "stacktrace"}}

1.3 Setup, Selenium Manager and your first script

Install JavaScript: npm install selenium-webdriver (Node 22+).

Selenium Manager Ships with Selenium since 4.6. A fallback that discovers the browser version, resolves, downloads and caches the driver in ~/.cache/selenium, and since 4.11 can download browsers.

Driver mismatch An old driver on PATH or given by hand gives session not created. Delete it and let Selenium Manager match the driver.

Eight components Start session, act on browser, ask the browser, wait strategy, find element, act on element, ask the element, end session.

```
const title = await driver.getTitle();
```

3. Request browser information: the page title.

```
await driver.manage().setTimeouts({ implicit: 500 });
```

4. Establish a waiting strategy. JavaScript takes milliseconds.

```
const textBox = await driver.findElement(By.name('my-text'));
```

5. Find an element by its name attribute.

```
const submitButton = await driver.findElement(By.css('button'));
```

5. Find the first button with a CSS selector.

```
await textBox.sendKeys('Selenium');
```

6. Take action on an element: type into the box.

```
await submitButton.click();
```

6. Click Submit. The browser loads the next page.

```
const message = await driver.findElement(By.id('message'));
```

Find the message on the new page.

```
const value = await message.getText();
```

7. Request element information: its visible text.

```
await driver.findElement(By.id('no-such-id'));
```

This element does not exist, so the promise rejects.

```
if (!(e instanceof error.NoSuchElementError)) throw e;
```

Only swallow the error we expected. Anything else is a real bug.

TRAP If you pass a driver path yourself, Selenium trusts you and never calls Selenium Manager. The documentation says that with Selenium 4.6 or newer you should not need to set a driver location at all. Delete old drivers from PATH and let Selenium Manager do the job.

TRAP The documentation itself warns that an implicit wait is rarely the best solution. It is used here because it is the easiest to show. Chapter 4.1 replaces it with explicit waits.

STEP 2 Control the browser

2.1 Sessions, options and timeouts

Options Since Selenium 4, describe sessions with Options classes. They become W3C capabilities, and browser specific ones carry a prefix such as goog:chromeOptions.

Page load strategy normal waits for complete (load), eager for interactive (DOMContentLoaded), none for the first download. It covers navigation by address, not clicks.

Default timeouts Implicit 0, page load 300,000 ms, script 30,000 ms.

Service Starts and stops a local driver: file, port, arguments and logs. Not used for remote sessions.

Granted capabilities Read them after New Session to learn the real browser version, platform and defaults such as dismiss and notify.

```
const chrome = require('selenium-webdriver/chrome');
```

The Chrome module holds chrome.Options.

```
const options = new chrome.Options();
```

An Options object describes the session you want.

```
options.addArguments('--headless=new', '--window-size=1280,800');
```

Chrome command line switches: no window, and a fixed size.

```
options.setPageLoadStrategy('eager');
```

A W3C capability: get() returns at DOMContentLoaded.

```
const driver = await new Builder().forBrowser(Browser.CHROME).setChromeOptions(options).build();
```

New Session is sent with these capabilities.

```
const caps = await driver.getCapabilities();
```
The capabilities the driver actually granted.

```
await driver.manage().setTimeouts({ script: 1000 });
```
Allow scripts only one second from now on.

```
await driver.executeAsyncScript('/* never calls its callback */');
```
An async script must call its callback. This one never does.

```
if (!(e instanceof error.ScriptTimeoutError)) throw e;
```
The W3C error script timeout becomes ScriptTimeoutError.

```
const service = new chrome.ServiceBuilder().setPort(9600);
```
A ServiceBuilder describes how to start chromedriver: here, on port 9600.

```
const driver = await new Builder().forBrowser(Browser.CHROME).setChromeService(service).build();
```
Selenium starts chromedriver on that port, then opens a session.

```
const reply = await fetch('http://localhost:9600/status');
```
GET /status is a W3C command that needs no session. fetch is built into Node.

Page load 300,000 ms

TRAP Switching to eager or none to make tests faster is a trade. Every element that arrives later now needs an explicit wait, or the test becomes flaky. Chapter 4.1 shows how to write those waits.

EXAM These timeouts belong to the browser session. Explicit waits such as WebDriverWait are different: they run in your code and poll the browser. Mixing an implicit wait with explicit waits can give surprising wait times, which chapter 4.1 explains.

2.2 Navigation, window size, screenshots and PDF

Navigation get or navigate().to, back, forward, refresh.

Where am I Title and current URL. Check them when a test depends on a redirect or on history, because back on the first page raises no error.

Window rect Set and get size and position. An exact size is more portable than maximize, which needs a window manager.

Screenshots Take Screenshot (viewport) and Take Element Screenshot return base64 PNG. Save one when a test fails, before quit.

Print Page Returns a base64 PDF. Use PrintOptions for orientation, ranges, size, margins and scale. Chromium needs headless.

```
await driver.navigate().to('https://www.selenium.dev/selenium/web/xhtmlTest.html');
```
The long form of get.

```
await driver.navigate().back();
```
Press the Back button.

```
await driver.navigate().forward();
```
Press the Forward button.

```
await driver.navigate().refresh();
```
Reload the current page.

```
const window = driver.manage().window();
```
Window commands live under manage().window().

```
await window.setRect({ width: 1024, height: 768, x: 40, y: 30 });
```
JavaScript sets size and position in one call to Set Window Rect.

```
const { width, height, x, y } = await window.getRect();
```
Get Window Rect returns all four numbers.

```
const pngSize = file => { const b = fs.readFileSync(file); return [b.readUInt32BE(16), b.readUInt32BE(20)]; };
```
A PNG keeps width and height at bytes 16 and 20.

```
const page = await driver.takeScreenshot();
```
Take Screenshot answers with the PNG as base64 text.

```
fs.writeFileSync('page.png', page, 'base64');
```
Decode the base64 and write the bytes to a file.

```
fs.writeFileSync('button.png', await button.takeScreenshot(), 'base64');
```
Take Element Screenshot: just the area of the button.

```
const options = new chrome.Options().addArguments('--headless=new');
```
Chromium prints to PDF only in headless mode.

```
const base64 = await driver.printPage({ orientation: 'landscape' });
```
The W3C Print Page command answers with base64 text.

```
const pdf = Buffer.from(base64, 'base64');
```
Decode it to the real bytes.

TRAP After refresh or any navigation, the elements you found before belong to the old page. Using them raises stale element reference. Find them again on the new page. Chapter 5.4 covers this error in depth.

TRAP Maximize and minimize need a desktop window manager. On a bare Linux build server they can fail with an unknown error, which is what happened when this page was built. Setting an exact size with Set Window Rect or the --window-size switch works everywhere and gives the same layout on every machine.

STEP 3 Find and use elements

3.1 Locators: how Selenium finds an element

Eight strategies id, name, class name, css selector, xpath, tag name, link text, partial link text. find element returns the first match.

Five on the wire W3C defines css selector, link text, partial link text, tag name and xpath. Bindings turn id, name and class name into CSS.

CSS and XPath CSS: #id, .class, tag[attr="v"], parent > child. XPath adds text matching and moving up to a parent. Prefer relative XPath (//).

Relative locators above, below, toLeftOf, toRightOf and near (50 px), built on getBoundingClientRect. They depend on layout, so check what they return.

Preference Unique stable id, then a short CSS selector, then XPath. Avoid absolute paths, styling classes and generated ids. Consider data-test attributes.

```
async function show(how, e) {
```
A helper that prints which element a locator found.

```
await show('className', await driver.findElement(By.className('information')));
```
The first element whose class list contains information.

```
await show('css', await driver.findElement(By.css('#fname')));
```
CSS: # means id, so the input with id fname.

```
await show('id', await driver.findElement(By.id('lname')));
```
The element whose id attribute is lname.

```
await show('name', await driver.findElement(By.name('newsletter')));
```
The element whose name attribute is newsletter.

```
await show('linkText', await driver.findElement(By.linkText('Selenium Official Page')));
```
A link whose whole visible text matches.

```
await show('partialLinkText', await driver.findElement(By.partialLinkText('Official')));
```
A link whose visible text contains Official.

```
await show('tagName', await driver.findElement(By.tagName('a')));
```
The first a element on the page.

```
await show('xpath', await driver.findElement(By.xpath("//input[@value='f']")));
```
XPath: any input whose value attribute is f.

```
await show('above lname', await driver.findElement(locateWith(By.tagName('input')).above(By.id('lname'))));
```
An input above the lname box.

TRAP Each relative locator sends the page a script to run, so it is slower than a plain CSS selector. Prefer an id or a short CSS selector, and keep relative locators for elements with no good attributes.

EXAM Keep locators in one place, apart from the steps that use them. The documentation recommends declaring them separately, which is exactly what a page object does. Chapter 5.2 shows the pattern.

3.2 Finding elements: one, many, scoped and in shadow DOM

One or many find element returns the first match or raises no such element. find elements returns all matches or an empty list.

Search context You can search from the driver, from an element or from a shadow root. CSS from an element stays inside it.

XPath trap An XPath starting with // always starts at the document, even when called on an element. Use .// to search below the element.

Shadow DOM Find the host, get its shadow root, then search with a CSS selector. Chrome refuses XPath there.

Active element switchTo().activeElement() returns the focused element. Use it to test keyboard flow such as Tab order.

```
const inputs = await driver.findElements(By.tagName('input'));
```
Plural: an array of every input, in page order.

```
for (const e of inputs) {
```
for...of with await inside, so the questions are asked one by one.

```
const none = await driver.findElements(By.id('email'));
```
No match: an empty array, never an error.

```
await driver.findElement(By.id('email'));
```
Singular with no match: the promise rejects.

```
const form = await driver.findElement(By.css('form'));
```
First find the parent element.

```
const last = await form.findElement(By.id('lname'));
```
Two commands: find the form, then find inside it.

```
const same = await driver.findElement(By.css('form #lname'));
```
The optimised version: one CSS selector, one command.

```
const host = await driver.findElement(By.css('custom-checkbox-element'));
```
The shadow host: the element the hidden tree is attached to.

```
const shadowRoot = await host.getShadowRoot();
```
Get Element Shadow Root.

```
const checkbox = await shadowRoot.findElement(By.css(box));
```
Search inside the shadow tree.

```
await shadowRoot.findElement(By.xpath('.//input'));
```
XPath inside a shadow root.

```
await driver.findElement(By.name('my-text')).click();
```
Clicking a text box gives it the focus.

```
await (await driver.switchTo().activeElement()).sendKeys(Key.TAB);
```
Press Tab on whatever has the focus, as a user would.

TRAP find elements never waits for the first element to appear unless an implicit wait is set, and then it returns as soon as at least one matches. An empty list can mean the page is not ready yet. Chapter 4.1 shows how to wait properly.

3.3 Interacting with elements and reading them

Five commands click, send keys, clear, submit and select. Prefer clicking the submit button over submit(). send keys appends, so clear first.

Built in checks Before acting the driver scrolls the element into view and checks it is interactable. Errors: element not interactable, invalid element state, element click intercepted.

Element information isDisplayed, isEnabled, isSelected, tag name, rect, CSS value and text. isDisplayed is a JavaScript approximation.

Attribute vs property DOM attribute is the HTML, DOM property is the live value. The classic getAttribute mixes both.

Select and upload Select works only on select elements: by visible text, value or index. Upload by sending the full path to input type="file".

```
await driver.manage().setTimeouts({ implicit: 2000 });
```
As in chapter 1.3: let finds retry for up to 2 seconds, so the page after Submit has time to load.

```
await text.sendKeys('first try');
```
Element Send Keys: types into the box, adding to what is already there.

```
await text.clear();
```
Element Clear: empties the box.

```
await text.sendKeys('Selenium');
```
Type the real value.

```
await check2.click();
```
Element Click: ticks it, as a mouse click would.

```
await driver.findElement(By.id('my-radio-2')).click();
```
Clicking the second radio button unselects the first one.

```
await driver.findElement(By.css('button')).click();
```
Click Submit. The documentation prefers this to the submit() method.

```
async function attempt(what, step) {
```
Run one step and report the error it raises, if any.

```
if (!(e instanceof error.WebDriverError)) throw e;
```
Every Selenium error extends WebDriverError.

```
await attempt('clear readonly', async () => (await driver.findElement(By.name('my-readonly'))).clear());
```
A readonly input cannot be reset.

```
await driver.executeScript(
```
Add a cookie banner that covers the bottom of the page, as many real sites do.

```
await attempt('click under banner', async () => (await driver.findElement(By.css('button'))).click());
```
The banner sits over the centre of the button.

```
await text.sendKeys('hello');
```
Type, so the live value differs from the HTML.

```
const rect = await text.getRect();
```
Get Element Rect: position and size in pixels.

```
const single = new Select(await driver.findElement(By.name('selectomatic')));
```
Wrap a select element in a Select object.

```
await single.selectByVisibleText('Four');
```
Pick by the text the user sees.

```
await single.selectByValue('two');
```
Pick by the value attribute.

```
await single.selectByIndex(0);
```
Pick by position, counting from 0.

```
const multi = new Select(await driver.findElement(By.id('multi')));
```
A select with the multiple attribute.

```
await multi.deselectByValue('eggs');
```
Only multiple lists can deselect.

```
fs.writeFileSync('report.txt', 'quarterly numbers');
```
Make a small file to upload.

```
const input = await driver.findElement(By.name('my-file'));
```
The input with type="file".

```
await input.sendKeys(path.resolve('report.txt'));
```
Send the full path. No dialog opens, the file is simply attached.

EXAM isDisplayed is not defined by the W3C spec, because deciding what a person can see covers too many cases. The documentation explains that Selenium runs a large JavaScript function that makes approximations. Treat it as a good guide, not a perfect one.

STEP 4 Synchronise and control

4.1 Waiting strategies: implicit, explicit and fluent

The race Content added or shown by JavaScript after load is not covered by navigation waits. Finding it too early fails. Do not use fixed sleeps.

Implicit wait A session setting, default 0. Every find keeps looking up to that long and returns as soon as the element appears. It only waits for presence.

Explicit wait A loop with a condition, timeout and polling interval. JavaScript waits forever without a timeout.

Expected conditions presence, visibility, clickable, text present, title contains, staleness. Wait for visibility before typing.

Never mix Implicit plus explicit makes waits unpredictable: 10 s with 15 s can time out after about 20 s. Keep implicit at 0 and use explicit waits.

```
await driver.findElement(By.id('adder')).click();
```
The page adds a red box, but only 1 second after the click.

```
await driver.findElement(By.id('box0'));
```
Our code asks at once. No wait is set, so the driver checks only once.

```
const box = await driver.findElement(By.id('box0'));
```
Not there yet, so the driver keeps checking until the box appears.

```
const revealed = await driver.findElement(By.id('revealed'));
```
The input is in the page already, just hidden.

```
await driver.findElement(By.id('reveal')).click();
```
The page shows it 1 second later.

```
await driver.wait(() => revealed.isDisplayed(), 2000);
```
Keep calling the function until it is true, for at most 2000 ms. JavaScript checks every 200 ms.

```
await revealed.sendKeys('Displayed');
```
Safe now: the input is displayed, so it can take keys.

```
const box = await driver.wait(until.elementLocated(By.id('box0')), 2000);
```
A ready made condition. It resolves to the element.

```
await driver.wait(until.elementIsVisible(field), 2000);
```
Present is not enough for typing. Wait until it is visible.

```
await driver.wait(until.elementLocated(By.id('box1')), 1000, 'the second box never came');
```
Nobody clicked again, so box1 never appears. The third argument is our message.

```
await driver.wait(async () => {
```
JavaScript has no ignore list, so the condition catches the error itself.

```
return true;
```
Worked: the wait ends.

```
if (e instanceof error.ElementNotInteractableError) return false;
```
Not yet: the wait tries again.

```
throw e;
```
Any other error is a real problem.

4.2 Windows, tabs, frames and alerts

Windows Each window or tab has a handle. Save the original, switch to the new one, and switch back after close(). newWindow("tab") opens and switches in one step.

Frames Switch into an iframe by element, name or id, or index before finding inside it. defaultContent goes to the top, parentFrame up one level.

Title in frames Get Title always reports the top level page, even from inside a frame.

Alerts alert, confirm and prompt: getText, accept, dismiss, sendKeys. Wait with alert is present when it appears late.
Forgotten pop ups unhandledPromptBehavior defaults to dismiss and notify: the pop up is closed and the next command raises unexpected alert open.

```
const original = await driver.getWindowHandle();
```
The id of the current window. It stays the same for the whole session.

```
await driver.findElement(By.name('windowTwo')).click();
```
This link has target="_blank", so it opens a new window.

```
await driver.wait(async () => (await driver.getAllWindowHandles()).length === 2, 2000);
```
Wait until the second window exists.

```
for (const handle of await driver.getAllWindowHandles()) {
```
Every open window, in the order they were opened.

```
if (handle !== original) await driver.switchTo().window(handle);
```
Switch to the one that is not the original.

```
await driver.close();
```
Close only this window.

```
await driver.switchTo().window(original);
```
Switch back, or the next command fails with no such window.

```
await driver.switchTo().newWindow('tab');
```
Selenium 4: open a new tab and switch to it in one step.

```
await driver.switchTo().frame(frame);
```
1. Switch with a WebElement: the most flexible way.

```
await driver.switchTo().defaultContent();
```
Leave the frame: back to the top document.

```
await driver.switchTo().frame(await driver.findElement(By.name('iframe1-name')));
```
2. JavaScript has no switch by name string, so find the frame by its name first.

```
await driver.switchTo().frame(0);
```
3. Switch with an index: the first frame of the page.

```
await driver.findElement(By.id('alert')).click();
```
The page calls alert('cheese').

```
const alert = await driver.wait(until.alertIsPresent(), 2000);
```
Wait for the alert, then get a handle on it.

```
await alert.accept();
```
Press OK.

```
await driver.findElement(By.id('confirm')).click();
```
A confirm box: OK or Cancel.

```
const confirm = await driver.switchTo().alert();
```
Switch to it without waiting, because it opens at once.

```
await confirm.dismiss();
```
Press Cancel.

```
await driver.findElement(By.id('prompt')).click();
```
A prompt asks for text.

```
await prompt.sendKeys('Selenium');
```
Type the answer into the prompt.

```
await driver.findElement(By.id('slow-alert')).click();
```
This one opens 200 ms later.

```
const slow = await driver.wait(until.alertIsPresent(), 2000);
```
switchTo().alert() now would fail, so wait.

```
await driver.getTitle();
```
Any command while an alert is open.

TRAP After close() you are still pointing at the closed window. The documentation warns that the next command raises no such window. Always switch back to a handle you saved before.

4.3 The Actions API: mouse, keyboard and wheel

Build then perform An action chain is sent in one request when perform() is called. Nothing happens before that.
Ticks Each input source has one action per tick. A tick lasts as long as its slowest action.
Mouse doubleClick, contextClick, moveToElement, clickAndHold, dragAndDrop, moveByOffset. The driver remembers the pointer position between chains.
Keyboard keyDown and keyUp for modifiers, sendKeys for text. Use Command on a Mac and Control elsewhere. Release every key you press.
Wheel scrollToElement, scrollByAmount, scrollFromOrigin. Wait for the scroll result, because it can land after perform() returns.

```
await driver.actions().doubleClick(box).perform();
```
Build a chain, then perform() sends it to the browser in one request.

```
await driver.actions().contextClick(box).perform();
```
A right click.

```
await driver.actions().move({ origin: await driver.findElement(By.id('hover')) }).perform();
```
Move to the centre of the element. Nothing is clicked.

```
await driver.actions().dragAndDrop(from, to).perform();
```
Move to from, press, move to to, release.

```
await driver.actions().move({ x: 8, y: 12 }).perform();
```
With no origin, x and y are measured from the top left of the viewport.

```
await driver.actions().move({ x: 13, y: 15, origin: Origin.POINTER }).perform();
```
From wherever the pointer is now. The driver remembered it between chains.

```
.keyDown(Key.SHIFT)
```
Press Shift and keep holding it.

```
.sendKeys('abc')
```
Goes to the element that has focus. The page focuses this box on load.

```
.keyUp(Key.SHIFT)
```
Let go of Shift.

```
const cmdCtrl = process.platform === 'darwin' ? Key.COMMAND : Key.CONTROL;
```
Select all is Command A on a Mac and Control A elsewhere.

```
await driver.actions().click(input).sendKeys('Selenium!').perform();
```
JavaScript has no send keys to element in actions, so click first, then type.

```
const frame = await driver.findElement(By.tagName('iframe'));
```
The page starts with a 5000 px tall block, so the iframe is far below.

```
await driver.actions().scroll(0, 0, 0, 0, frame).perform();
```
scroll(x, y, deltaX, deltaY, origin). With an element as origin and no delta, it scrolls the element into view.

```
await driver.actions().scroll(0, 0, 0, 300).perform();
```
Like turning the wheel: 300 px down from the top left of the viewport.

```
await driver.wait(async () => (await driver.executeScript('return window.scrollY')) >= 300, 2000);
```
The scroll can land after perform returns, so wait for it.

```
await driver.actions().scroll(0, 0, 0, 200, inner).perform();
```
Turn the wheel with the pointer over the frame, so the frame scrolls.

TRAP Actions type into the focused element. If nothing has focus, the keys go to the page body and the test seems to type into nothing. Element sendKeys focuses the element itself, which is why it is the better choice for plain typing.

4.4 JavaScript execution and cookies

executeScript The script runs as a function body. Use return to get a value and arguments[n] to read what you pass in. Elements go in and come back as WebElements.
Async scripts executeAsyncScript adds a callback as the last argument. A returned promise is also awaited. The script timeout, 30 s by default, limits both.
JavaScript click It skips the checks a real click makes, so use it only when you know why the real click cannot work.
Cookies Visit the domain first, then add. httpOnly hides a cookie from page scripts. deleteAllCookies gives a clean start.

```
await driver.executeScript('arguments[0].click();', await driver.findElement(By.id('updatediv')));
```
A click done by JavaScript. It skips the checks a real click makes.

```
await driver.manage().setTimeouts({ script: 200 });
```
How long a script may run, in ms. The default is 30000.

```
await driver.manage().addCookie({ name: 'key', value: 'value' });
```
The browser is still on an empty start page with no domain.

```
await driver.get('https://www.selenium.dev/selenium/web/blank.html');
```
Visit the domain first. A small page is enough.

```
const c = await driver.manage().getCookie('key');
```
Read one cookie by name. It comes back as an object.

```
await driver.manage().addCookie({ name: 'foo', value: 'bar', sameSite: 'Strict' });
```
Any other field goes in the same object.

```
await driver.manage().addCookie({ name: 'secret', value: 's', httpOnly: true });
```
httpOnly hides the cookie from page JavaScript.

```
await driver.manage().deleteAllCookies();
```
A clean start, for example before logging in as another user.

TRAP A click done with JavaScript always works, even on a covered or disabled element, because it skips every check a user faces. A test that passes with a JavaScript click may be hiding a real bug. Use it only when you know why the real click cannot work.

STEP 5 Build a test suite

5.1 Test frameworks: JUnit 5, pytest and Mocha

Framework jobs Selenium drives the browser.

Fresh browser Start the driver in the before each step and quit it in the after each step, so tests stay independent and can run in any order.

Teardown always runs @AfterEach, code after yield and afterEach run also when the test failed, so quit belongs there.

Data-driven @ParameterizedTest with @CsvSource, @pytest.mark.parametrize, or a loop around it() in Mocha. One result per row.

Screenshot on failure Take it after the test failed and before quit: Mocha this.currentTest.state.

```
describe('Web form', function () {
```
describe groups tests. Mocha gives every test a name.
```
beforeEach(async function () {
```
Runs before every test.
```
it('has the right title', async function () {
```
One test.
```
assert.strictEqual(await driver.getTitle(), 'Web form');
```
Node's own assert module. Actual first, then expected.
```
afterEach(async function () {
```
Runs after every test, also when the test failed.
```
const rows = [
```
Each row is one run of the test.
```
for (const [user, password, expected] of rows) {
```
Mocha has no data annotation. A loop that calls it() once per row does the same job.
```
assert.strictEqual(await driver.getTitle(), 'Web form!');
```
Wrong on purpose, so the test fails.
```
if (this.currentTest.state === 'failed') {
```
Mocha tells the hook how the test ended.
```
fs.writeFileSync(name, await driver.takeScreenshot(), 'base64');
```
Take the picture before quit.

5.2 Page objects and page components

Page object A class that is the interface to one page. Tests call its methods, so a change to the page is fixed in one place.

No assertions inside Assertions belong to the test. The only check in a page object is that the right page is loaded.

Return the next page A method that moves to another page returns that page object. A method that stays may return itself.

Components A page component object models a repeated part, such as a field or a product card, and searches only inside its root element.

Waits inside Put waits in the page object, for example a constructor that waits for its title after a navigation.
```
const FIELDS = By.css('label.form-label');
```
Locators live here and nowhere else.
```
static async at(driver) {
```
A constructor cannot await, so an async factory does the page check.
```
return (await this.driver.findElements(FIELDS)).map(e => new Field(e));
```
Wrap every label element in a Field component.
```
class Field {
```
A page component: one label and the control inside it.
```
async name() { return (await this.root.getText()).split('\n')[0].trim(); }
```
The first line of the label is its name.
```
control() { return this.root.findElement(By.css('input, textarea, select')); }
```
Search inside this component only.
```
await driver.wait(until.titleIs('Web form - target page'), 3000);
```
The page arrives after a navigation, so wait for it instead of checking once.
```
const form = await WebFormPage.open(driver);
```
The test talks to pages, not to locators.
```
const received = await form.submit();
```
submit() moves to another page, so it returns that page object.
```
assert.strictEqual(await received.message(), 'Received!');
```
The assertion stays in the test.
```
assert.strictEqual(fields.length, 11);
```
Eleven labelled controls on this page.
```
await assert.rejects(WebFormPage.at(driver), /This is not the web form/);
```
The check fails loudly.

5.3 Good practices and things to avoid

Need a browser? Ask first whether a lighter test can do the job. Browser tests are the most expensive kind.

Short and focused Set up the data, perform a few actions, evaluate the result. Many small independent tests beat one grand journey.

Clean start A new driver starts with a new profile. Use a fresh browser per test and never share test data.

Prepare state outside the UI Log in and create data through APIs, then set cookies, instead of clicking through forms.

Avoid CAPTCHAs, download progress, status codes, third party logins, test order, performance numbers, spidering and 2FA are jobs for other tools or test hooks.
```
await first.manage().addCookie({ name: 'cart', value: '3 items' });
```
State a first test leaves behind: a cookie
```
await first.executeScript("localStorage.setItem('theme', 'dark')");
```
and a value in local storage.
```
const second = await new Builder().forBrowser(Browser.CHROME).build();
```
A new driver gets a new, empty profile by default.
```
const session = 'abc123';
```
In a real suite this value comes from a login API call, not from the login form.
```
await driver.get('https://www.selenium.dev/selenium/web/cookies.html');
```
Now open the page under test. It reads its cookies on load.

5.4 Troubleshooting: errors, stale elements and logs

Error names Each W3C error has a class per language. Read the name, then the message, then your line in the trace.

Stale element The node behind a reference was replaced, for example by a refresh. Find the element again.

Context An element found in one frame or window is unknown in another. Switch back before using it.

Client log JavaScript 4.51: driver.http.Executor at FINER.

Driver log ChromeDriver writes COMMAND and RESPONSE lines when its service is given a log file and a verbose level.
```
let box = await driver.findElement(By.name('my-text'));
```
The driver gives back a reference to this exact node.
```
box = await driver.findElement(By.name('my-text'));
```
The fix: find it again.
```
const email = await driver.findElement(By.id('email'));
```
Found inside the frame.
```
const http = logging.getLogger('driver.http.Executor');
```
In selenium-webdriver 4.51 the HTTP lines come from this logger.
```
http.setLevel(logging.Level.FINER);
```
Request lines are written at FINER.
```
http.addHandler(entry => {
```
A tiny handler that prints one line per request.
```
const m = /^\[[\w.]+\] >>> (\w+) (\S+)$/.exec(entry.message);
```
Each message starts with the logger name in square brackets.
```
if (m) console.log(m[1] + ' ' + m[2].replace(/\/session\/[0-9a-f]{32}/, '/session/{id}'));
```
Hide the session id, which changes every run.
```
const service = new chrome.ServiceBuilder().loggingTo('chromedriver.log').enableVerboseLogging();
```
Where ChromeDriver writes its log, with full detail.
```
if (line.includes('] COMMAND ')) console.log(line.split('] COMMAND ')[1].split(' ')[0]);
```
Print the name of each command the driver received.

TRAP The documentation shows the JavaScript logger as getLogger("webdriver"). In selenium-webdriver 4.51 that name printed nothing in our run, while driver.http and driver.http.Executor printed every request.

STEP 6 Scale out with Grid

6.1 Selenium Grid 4

Why a Grid Parallel runs on many browsers, versions and systems, and shorter suites: tests × average time ÷ nodes.

Components Router (4444), New Session Queue, Distributor, Session Map, Node (5555) and Event Bus (4442, 4443).

Roles Standalone for one machine, Hub and Node for most Grids, Distributed for very large ones.

Remote driver Only the driver creation changes. se:name adds metadata, se:cdp gives DevTools through the Grid, and a local file detector sends upload files to the node.

Queue and quit New sessions wait in the queue for a free slot. Always quit, or the slot stays busy until the session timeout.

```
const res = await fetch('http://localhost:4444/status');
```

The same endpoint a driver has, now answered by the Grid.

```
const driver = await new Builder().forBrowser(Browser.CHROME).usingServer('http://localhost:4444').build();
```

Resolves when the Grid has given this test a slot.

```
await new Promise(r => setTimeout(r, 3000));
```

Pretend the test takes 3 seconds.

```
const times = (await Promise.all([oneTest(), oneTest(), oneTest()])).sort((a, b) => a - b);
```

Three tests start at the same moment.

```
options.set('se:name', 'upload on the grid');
```

se: capabilities are Grid metadata. se:name shows in the Grid UI.

```
driver.setFileDetector(new remote.FileDetector());
```

Sends the local file to the node, which may be another machine.

Suite time on a Grid

$$\text{total time} = \frac{\text{number of tests} \times \text{average test time}}{\text{number of nodes}}$$

6.2 WebDriver BiDi, CDP and Grid

features

BiDi A W3C protocol with a WebSocket, so the browser can push events. Turn it on with webSocketUrl, then subscribe to console and page errors.

Events, not polling Handlers are called when events arrive. Wait for them, because they come after the action that caused them.

CDP Chrome and Edge only, no stability promise, the three newest Chrome versions. Use it for Chromium features with no other API.

GraphQL POST a query to /graphql on the Grid for nodes, slots, sessions and the queue size.

Grid endpoints /status, DELETE /session, and drain or remove a node through the Distributor endpoints.

```
const inspector = await LogInspector(driver);
```

Subscribes to log events.

```
await inspector.onConsoleEntry(e => heard.push(`console ${e.level}: ${e.text}`));
```

Called for every console.log, console.error and so on.

```
await inspector.onJavascriptException(e => heard.push(`page error: ${e.text}`));
```

Called for every uncaught error in the page.

```
await driver.wait(() => heard.length === 3, 5000);
```

Events are pushed, not returned, so wait until all three arrived.

```
const version = await driver.sendAndGetDevToolsCommand('Browser.getVersion', {});
```

Any CDP method by name, with an object of parameters.

```
await driver.sendAndGetDevToolsCommand('Emulation.setTimezoneOverride', { timezoneId: 'Asia/Kolkata' });
```

Something WebDriver Classic has no command for.

```
const res = await fetch('http://localhost:4444/graphql', { method: 'POST', body: JSON.stringify({ query }) });
```

GraphQL over HTTP: one POST with the query as a string.

```
const q = '{ grid { nodeCount, totalSlots, maxSession, sessionCount, sessionQueueSize } }';
```

Ask only for these fields.

STEP 7 Revise and practise

7.1 Revision: predict, debug and design

Predict, then run The driver does what the protocol says, not what the code seems to say. Offsets add, Shift changes case, a refresh makes references stale.

Cause before fix Name why a test is slow or flaky before changing it. Sleeps, mixed waits and missing quit calls are the usual causes.

Layers Tests, page objects and components, a driver factory and support code. Each change should touch one layer.

Size with the formula Suite time = tests × average time ÷ nodes, with about one CPU and 1 GB of RAM per browser.

Know your binding Time is a Duration in Java, seconds in Python and milliseconds in JavaScript, and a missing cookie gives null, None or an error.

Implicit wait 0

TRAP A good bug answer has three parts: what the code really does, why that breaks, and a fix that removes the cause. A retry that hides the failure is not a fix.